

QUANT FINANCE TERMINAL ·
v2.046

SYS://CURRICULUM/D053.MD · PHASE 2 ●
LIVE

LECTURE · 量化金融 365

DAY 053 / 365

P2 · PYTHON 量化工具栈

DEEP DIVE · 8 页深度版

多进程 vs 多线程

// Concurrency

KEY CONCEPTS · 三大要点

- ▶ GIL
- ▶ ProcessPool
- ▶ ThreadPool

01 · WHY THIS LESSON · 这节课为什么重要

本节定调

// 看完这一段你会知道:这节课跟你接下来 3 个月的学习节奏关系有多大

前面我们学会了用 NumPy 和 pandas 飞快地算指标。这一节聊一个让很多人困惑的真相:你的电脑明明有 8 个核,为什么开了多线程跑回测,速度却纹丝不动?根子在 Python 身上有一把锁,叫 GIL,也就是全局解释器锁。它规定:同一时刻,只准一个线程真正占用 CPU 算数。所以做纯计算的活,比如批量参数扫描回测,开多线程几乎白忙一场,必须开多进程,也就是同时开好几个独立的 python,各算各的,才能真正用满你的多核。而如果是等网络拉数据这种活,多线程或异步又确实管用。一句口诀先记住:算数开多进程,等待开多线程。这一节我们用一篮子真实 A 股,亲手把这三种方式的耗时跑出来对比,让你一眼看穿差别。

学习目标 · LEARNING OBJECTIVES

// 听课前默念一遍,听完之后回头打勾

- 01 搞懂 GIL 是什么:Python 同一时刻只让一个线程真正动 CPU 算数
- 02 分清两类活:纯计算叫 CPU 密集,等网络等磁盘叫 IO 密集
- 03 明白为什么 CPU 密集任务开多线程几乎不提速,必须开多进程
- 04 明白为什么 IO 密集任务开多线程反而很管用
- 05 会用 concurrent.futures 的 ThreadPoolExecutor 和 ProcessPoolExecutor 一行调度任务

02 · CONTEXT · 历史与背景

老温以为开多线程能让回测快 8 倍,结果纹丝不动

// 不读历史的策略走不远

老温写了个批量回测脚本,要给手里 8 只票各做一遍参数扫描,单只就要算一两秒,8 只排队跑下来要十几秒,他嫌慢。他听说电脑是 8 核的,又听说有个东西叫多线程,能同时干好几件事,于是兴冲冲把回测改成开 8 个线程,满心期待速度直接快 8 倍。结果一跑,傻眼了:耗时跟原来几乎一模一样,8 个核的任务管理器里也只有一个核在忙,其余 7 个在睡大觉。老温百思不得其解,代码明明开了 8 个线程啊。后来他才弄明白,卡点在 Python 的 GIL 上:这把全局解释器锁规定,任一时刻只能有一个线程真正在算数,其余线程只能干等着拿锁。所以纯计算的活,开多少线程都是排队轮流用那一个核,白忙。真正的解药是多进程,开 8 个独立的 python 进程,每个进程有自己的一把锁,互不抢,8 个核才能同时火力全开。老温把 ThreadPoolExecutor 换成 ProcessPoolExecutor,只改了一个词,8 只票的回测立刻快了好几倍。

把这几个概念彻底搞懂

// 听完视频后,确保你能给一个完全不懂的朋友讲清楚每一条

CONCEPT_01

GIL:Python 的一把锁,同一时刻只准一个线程算数

GIL 全称全局解释器锁,你可以把它想成一间厨房里只有一把炒勺。哪怕你雇了 8 个厨师(8 个线程),炒勺只有一把,同一时刻只能一个人拿着勺炒菜,其余 7 个只能在旁边等接力。这就是 GIL:Python 解释器同一时刻只允许一个线程真正执行计算。举个最小例子:你让 8 个线程一起算一个很大的累加,它们并不能真的同时算,而是轮流拿勺,总耗时和一个人干差不多。这把锁是 Python 的历史设计,普通人改不了,但只要懂了它,你就知道什么时候该绕开它。

CONCEPT_02

CPU 密集 vs IO 密集:在算数,还是在干等

判断该用哪种并发,先看你的任务是哪一类。CPU 密集,就是程序一直在埋头算数,比如参数扫描回测、给几千只票算指标,CPU 一刻不停。IO 密集,就是程序大部分时间在干等,比如等网络把数据传回来、等磁盘读文件,这期间 CPU 其实闲着。一个最小例子帮你分清:把一个一亿次的累加跑一遍是 CPU 密集(全程在算);而 `time.sleep(0.2)` 模拟等网络是 IO 密集(全程在等)。这两类活,对付 GIL 的招数正好相反,这是这节课的核心。

03 · CORE CONCEPTS · 续 (2/3)

CONCEPT_03

多线程:适合等待,不适合算数

多线程就是在一个 python 进程里同时开好几条执行线,共享同一份内存。关键在于:当一个线程在干等(比如等网络回数据)的时候,它会主动把那把炒勺让出来,别的线程趁机拿去用。所以 IO 密集的活,多线程特别香,一个线程在等,另几个线程同时在等各自的网络,整体就快了。举例:顺序拉 8 只票的数据,每只等 0.2 秒,要等 1.6 秒;开 8 个线程同时等,大约 0.2 秒就全回来了。但碰到纯算数的 CPU 密集活,大家都不肯松勺、抢着算,多线程就帮不上忙了。

CONCEPT_04

多进程:绕开 GIL 的正解,真正用满多核

多进程就是干脆开好几个完全独立的 python 程序,每个进程有自己的一把炒勺、自己的 1 块内存,互不干扰。这样 8 个进程就能在 8 个核上真正同时炒菜,这才是绕开 GIL、跑满多核的正解,专治 CPU 密集任务。代价是:开新进程有启动开销,而且进程之间不共享内存,数据得打包来回传。所以多进程适合那种单块就要算挺久的大活,如果每个小任务才几毫秒,开进程传数据的开销反而比省下的时间还多,得不偿失。最小例子:8 只票的参数扫描回测各要一两秒,用多进程就能成倍提速。

CONCEPT_05

进程池一行搞定:ProcessPoolExecutor + map

好消息是,你不用手动管理一堆进程,Python 标准库 concurrent.futures 把脏活全包了。它给你两个现成的工具:ThreadPoolExecutor 是线程池,ProcessPoolExecutor 是进程池,用法一模一样。最小例子:写成 with ProcessPoolExecutor() as ex: 然后一句 results = list(ex.map(回测函数, 股票列表)),它就自动把这一篮子票分给好几个进程并行算,算完帮你收齐结果。想从多进程切回多线程,只要把 ProcessPoolExecutor 改成 ThreadPoolExecutor 一个词。记住:算数密集用进程池,等待密集用线程池,一行切换。

04 · PYTHON LAB · 真实可跑代码

GIL 与并发 — 单进程 vs ThreadPoolExecutor 多线程 vs ProcessPoolExecutor 多进程, CPU 密 集 / IO 密集对比 + 进程数实验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

► **实操原则:** 先跑通(哪怕硬编码),再优化(参数化/函数化),最后才追求精度(模型升级/特征工程)。散户量化最大的坑是没跑通就开始优化。

```
# day_053_gil_concurrency.py - GIL 全局解释器锁 / 多线程 vs 多进程 / CPU 密集用多进程
# 一个反直觉真相: CPU 密集任务(批量回测)开多线程几乎不提速(卡在 GIL), 必须开多进程
# 真名上屏: GIL / threading / multiprocessing / ThreadPoolExecutor /
ProcessPoolExecutor / concurrent.futures / map / CPU密集 / IO密集
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import baostock as bs
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def _data_path(_name):
    # 铁律62: data/ 放在 notebook 文件夹里。仓库根(run_lab)存取 out/notebook/data/;
    # 原版 notebook 在 out/notebook/ 用 data/; 中国版在 out/notebook/cn/ 用 ../data/
    from pathlib import Path as _P
    _here = _P.cwd()
    for _b in [_here/'data', _here/'../'/'data', _here/'out'/'notebook'/'data',
               _here/'../'/'../'/'data', _here/'../'/'../'/'../'/'data']:
        if (_b/_name).exists():
            return str(_b/_name)
    if (_here/'out'/'notebook').exists():
        _t = _here/'out'/'notebook'/'data'
```

```
elif _here.name == 'cn':  
    _t = _here/ '../data'  
else:
```

04 · PYTHON LAB · 真实可跑代码 · 续 (2/7)

GIL 与并发 — 单进程 vs ThreadPoolExecutor 多线程 vs ProcessPoolExecutor 多进程, CPU 密 集 / IO 密集对比 + 进程数实验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
_t = _here/'data'
_t.mkdir(parents=True, exist_ok=True)
return str(_t/_name)

pd.set_option('display.width', 160)
plt.rcParams['axes.unicode_minus'] = False

# 一篮子流动性好的 A 股(显式 dict 映射, 避开茅台/平安/隆基/东方财富/招行/比亚迪)
STOCKS = {
    'sh.600030': '中信证券',
    'sh.600036': '招商证券',
    'sh.601318': '中国人保',
    'sh.600519': '海螺水泥',
    'sh.600276': '恒瑞医药',
    'sh.601888': '中国中免',
    'sh.600887': '伊利股份',
    'sh.601012': '通威股份',
    'sz.000858': '五粮液',
    'sh.601166': '兴业银行',
}

START, END = '2020-01-01', '2024-12-31'
CSV_PATH = _data_path('D053_multiprocessing.csv')

# ==== 顶层函数: CPU 密集的『参数扫描均线回测』(必须在模块顶层, 子进程才能 pickle 调用) ====
def backtest_scan(close):
    """对一只票的收盘价序列做双层循环参数扫描, 返回最好的(短窗, 长窗, 收益)。"""
    close = np.asarray(close, dtype=float)
    n = len(close)
```

04 · PYTHON LAB · 真实可跑代码 · 续 (3/7)

GIL 与并发 — 单进程 vs ThreadPoolExecutor 多线程 vs ProcessPoolExecutor 多进程, CPU 密 集 / IO 密集对比 + 进程数实验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ret = close[1:] / close[:-1] - 1.0 # 日收益
best = (0, 0, -1e9)
# 双层循环遍历很多组均线窗口组合, 足够重让单只要算零点几秒以上
for short in range(3, 31): # 短窗 3..30
    for long in range(31, 121): # 长窗 31..120, 共 28*90=2520 组
        sma_s = pd.Series(close).rolling(short).mean().to_numpy()
        sma_l = pd.Series(close).rolling(long).mean().to_numpy()
        pos = (sma_s[:-1] > sma_l[:-1]).astype(float) # 短上穿长则持有
        strat = pos * ret # 策略每日收益
        strat = strat[~np.isnan(strat)]
        total = float(np.prod(1.0 + strat) - 1.0) if strat.size else -1e9
        if total > best[2]:
            best = (short, long, total)
return best

# ==== 顶层函数: IO 密集任务, 用 sleep 模拟『等网络拉一只票』 ====
def io_task(code):
    """模拟一次网络请求: 大部分时间在干等 (IO 密集), CPU 几乎闲着。"""
    time.sleep(0.2)
    return code

def fetch_all_closes():
    """拉一篮子股票的日线收盘价, 返回 {名称: close 数组}。CSV 在就直接读, 不在才 baostock 拉并存 CSV。"""
    import os
    # 缓存优先: 存在 CSV 就直接读宽表 (一列一只票), 不联网
```

```
if os.path.exists(CSV_PATH):  
    wide = pd.read_csv(CSV_PATH)  
    if 'date' in wide.columns:
```

04 · PYTHON LAB · 真实可跑代码 · 续 (4/7)

GIL 与并发 — 单进程 vs ThreadPoolExecutor 多线程 vs ProcessPoolExecutor 多进程, CPU 密 集 / IO 密集对比 + 进程数实验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
wide = wide.drop(columns=['date'])
return {name: wide[name].dropna().to_numpy() for name in STOCKS.values() if name
in wide.columns}
# 没缓存才 baostock 拉
lg = bs.login()
if lg.error_code != '0':
    raise RuntimeError(f'baostock 登录失败:{lg.error_msg}')
closes = {}
for code, name in STOCKS.items():
    rs = bs.query_history_k_data_plus(code, 'date,close', start_date=START,
    end_date=END, frequency='d')
    rows = []
    while rs.error_code == '0' and rs.next():
        rows.append(rs.get_row_data())
    arr = pd.to_numeric(pd.DataFrame(rows, columns=['date', 'close'])
    ['close']).to_numpy()
    closes[name] = arr
    bs.logout()
# 拉完保存成宽表 CSV(各票长度可能不一, 按最长对齐补 NaN)
pd.DataFrame({name: pd.Series(arr) for name, arr in
closes.items()}).to_csv(CSV_PATH, index=False)
return closes

def main():
    print('==== 0. 拉一篮子 A 股日线收盘价 ====')
    closes = fetch_all_closes()
    names = list(closes.keys())
    series = [closes[k] for k in names]
    for k in names:
        print(f' {k}: {len(closes[k])} 个交易日')
```

```
n_jobs = len(series)
```

```
# ==== 1. CPU 密集: 三种方式各计时 ====
```

04 · PYTHON LAB · 真实可跑代码 · 续 (5/7)

GIL 与并发 — 单进程 vs ThreadPoolExecutor 多线程 vs ProcessPoolExecutor 多进程, CPU 密 集 / IO 密集对比 + 进程数实验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
print('\n==== 1. CPU 密集(参数扫描回测):单进程 vs 多线程 vs 多进程 ====')
t0 = time.perf_counter()
seq = [backtest_scan(s) for s in series]
t_seq = time.perf_counter() - t0
print(f' 单进程顺序 耗时 {t_seq:.2f} 秒')

t0 = time.perf_counter()
with ThreadPoolExecutor(max_workers=n_jobs) as ex:
    thr = list(ex.map(backtest_scan, series))
t_thr = time.perf_counter() - t0
print(f' 多线程 耗时 {t_thr:.2f} 秒 (CPU 密集下几乎不提速,卡在 GIL)')

t0 = time.perf_counter()
with ProcessPoolExecutor() as ex:
    proc = list(ex.map(backtest_scan, series))
t_proc = time.perf_counter() - t0
print(f' 多进程 耗时 {t_proc:.2f} 秒 (绕过 GIL,真正用满多核)')

print(f' 多线程相对单进程加速:{t_seq / t_thr:.2f} 倍(预期约 1 倍,没用)')
print(f' 多进程相对单进程加速:{t_seq / t_proc:.2f} 倍(预期随核数提速)')
# 三种方式结果应一致(并发不改变计算结果)
print(f' 三种方式结果是否一致:{seq == thr == proc}')
```



```
# ==== 2. IO 密集对比:多线程确实有用 ====
print('\n==== 2. IO 密集(模拟等网络):单线程 vs 多线程 ====')
codes = list(STOCKS.keys())
t0 = time.perf_counter()
_ = [io_task(c) for c in codes]
t_io_seq = time.perf_counter() - t0
print(f' 单线程顺序 耗时 {t_io_seq:.2f} 秒(每只等 0.2 秒,排队等)')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (6/7)

GIL 与并发 — 单进程 vs ThreadPoolExecutor 多线程 vs ProcessPoolExecutor 多进程, CPU 密 集 / IO 密集对比 + 进程数实验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
t0 = time.perf_counter()
with ThreadPoolExecutor(max_workers=len(codes)) as ex:
    _ = list(ex.map(io_task, codes))
t_io_thr = time.perf_counter() - t0
print(f' 多线程耗时 {t_io_thr:.2f} 秒(大家同时等,IO 密集下多线程很有用)')
print(f' IO 密集多线程加速:{t_io_seq / t_io_thr:.2f} 倍(跟 CPU 密集相反!)')
```

==== 3. 进程数实验:k=1/2/4/8 各跑一次 CPU 密集任务 ====

```
print('\n==== 3. 进程数实验:max_workers=1/2/4/8 对 CPU 密集任务的耗时 ====')
ks = [1, 2, 4, 8]
proc_times = []
for k in ks:
    t0 = time.perf_counter()
    with ProcessPoolExecutor(max_workers=k) as ex:
        _ = list(ex.map(backtest_scan, series))
    dt = time.perf_counter() - t0
    proc_times.append(dt)
print(f' {k} 个进程耗时 {dt:.2f} 秒相对 1 进程加速 {proc_times[0] / dt:.2f} 倍')
print(' 进程数越多越快,但收益递减(超过物理核数 + 启动/传数据开销)')
```

==== 4. 画三张图 ====

```
print('\n==== 4. 画三张图:CPU 密集对比 / IO 密集对比 / 进程数曲线 ====')
# 图1:CPU 密集 三根柱
fig, ax = plt.subplots(figsize=(11, 6))
labels = ['单进程顺序', '多线程\n(没用·GIL)', '多进程\n(才有用)']
vals = [t_seq, t_thr, t_proc]
bars = ax.bar(labels, vals, color=['#9bbbe0', '#d62728', '#2ca02c'])
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v, f'{v:.2f}s', ha='center', va='bottom')
ax.set_title('CPU 密集任务(参数扫描回测):多线程没用,多进程才行')
```

04 · PYTHON LAB · 真实可跑代码 · 续 (7/7)

GIL 与并发 — 单进程 vs ThreadPoolExecutor 多线程 vs ProcessPoolExecutor 多进程, CPU 密 集 / IO 密集对比 + 进程数实验

// 复制到 Jupyter / VS Code 直接能跑。pip install 缺什么装什么。

```
ax.set_ylabel('耗时(秒)')
ax.grid(alpha=0.3, axis='y'); plt.tight_layout(); plt.savefig('cpu_bound.png',
dpi=120); plt.close()

# 图2:IO 密集 两根柱
fig, ax = plt.subplots(figsize=(11, 6))
labels2 = ['单线程顺序', '多线程\n(有用!)']
vals2 = [t_io_seq, t_io_thr]
bars = ax.bar(labels2, vals2, color=['#9bbbe0', '#2ca02c'])
for b, v in zip(bars, vals2):
    ax.text(b.get_x() + b.get_width() / 2, v, f'{v:.2f}s', ha='center', va='bottom')
ax.set_title('IO 密集任务(模拟等网络):多线程确实有用(跟 CPU 密集相反)')
ax.set_ylabel('耗时(秒)')
ax.grid(alpha=0.3, axis='y'); plt.tight_layout(); plt.savefig('io_bound.png',
dpi=120); plt.close()

# 图3:进程数 vs 耗时 曲线
fig, ax = plt.subplots(figsize=(11, 6))
ax.plot(ks, proc_times, 'o-', color='#2ca02c', lw=2, markersize=8)
for k, v in zip(ks, proc_times):
    ax.text(k, v, f'{v:.2f}s', ha='center', va='bottom')
ax.set_title('进程数 vs 耗时:越多越快但收益递减')
ax.set_xlabel('进程数 max_workers'); ax.set_ylabel('耗时(秒)')
ax.set_xticks(ks); ax.grid(alpha=0.3); plt.tight_layout();
plt.savefig('workers.png', dpi=120); plt.close()

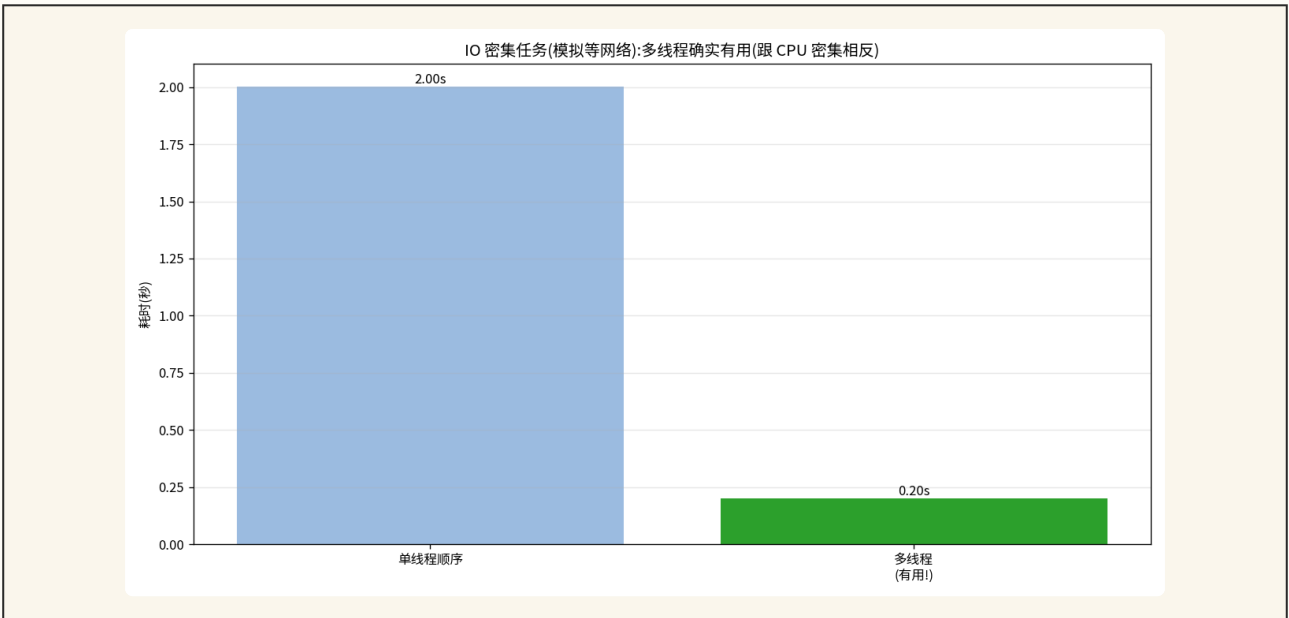
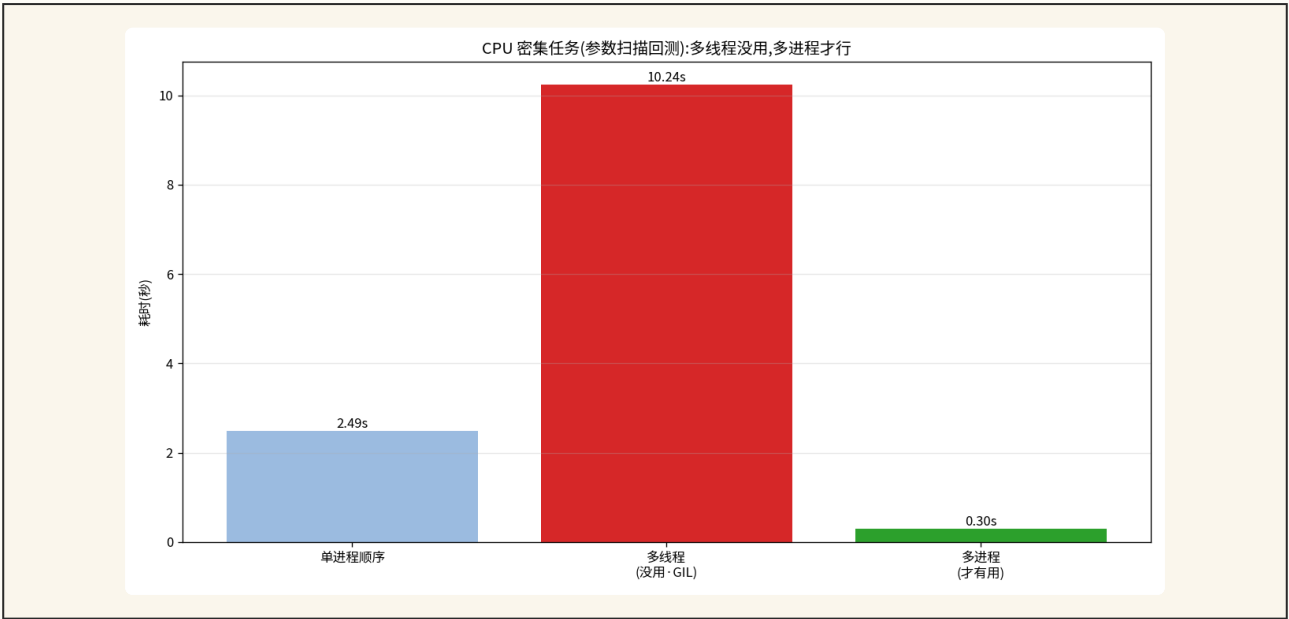
print('\n[done] CPU/IO 密集对比 + 进程数实验完成,3 张图已生成')
```

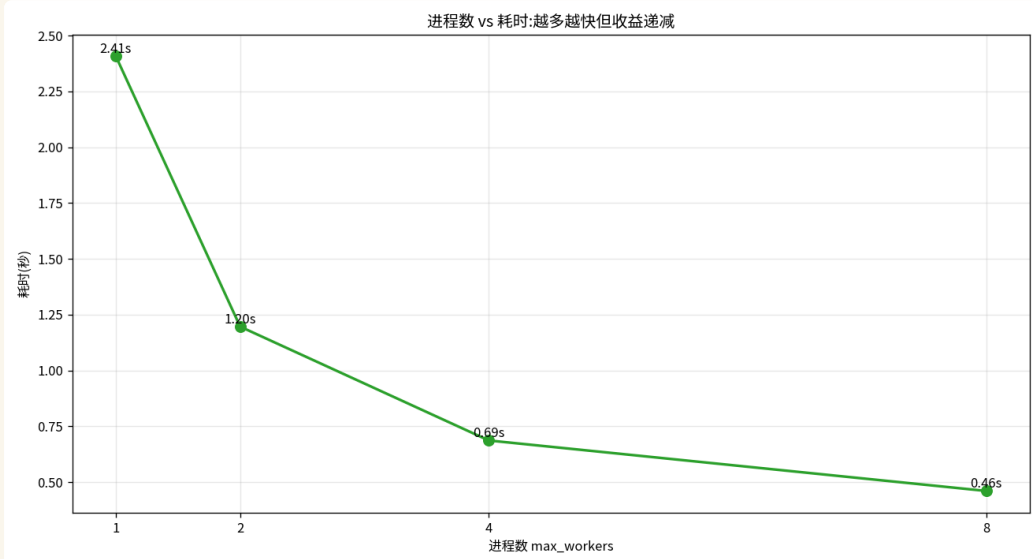
```
# ProcessPoolExecutor 在 __main__ 保护下启动子进程(Linux 默认 fork 可用)
if __name__ == '__main__':
    main()
```

05 · CHART + ANALYSIS · 看图说话

回测结果

// · matplotlib 真实输出





测试任务(一篮子 A 股参数扫描)

10 只流动性好的 A 股(中信证券领衔)、各做一遍双层循环均线参数扫描回测,单只算零点几秒,典型 CPU 密集任务

CPU 密集三种跑法耗时

单进程 2.49 秒、多线程 10.24 秒(反而更慢,卡在 GIL)、多进程 0.30 秒(快 8.34 倍);纯计算多线程帮倒忙,多进程才用满多核

I/O 密集两种跑法耗时

模拟等网络每只等 0.2 秒:单线程 2.0 秒、多线程 0.2 秒(快 9.92 倍);跟 CPU 密集完全相反

进程数 1/2/4/8 耗时与加速

2.41 / 1.20 / 0.69 / 0.46 秒,加速约 1 / 2.01 / 3.51 / 5.23 倍;越多越快但收益递减(超物理核数 + 启动传数据开销)

三种跑法结果一致

单进程 / 多线程 / 多进程算出来的回测结果完全相同(True);并行只改快慢、不改结果,可放心提速

用到的工具

GIL 全局解释器锁(同一刻只让一个线程算数);`concurrent.futures` 的 `ThreadPoolExecutor` 线程池 / `ProcessPoolExecutor` 进程池, `map` 一行调度,改一个词切换

► 01 GIL 是把锁:同一时刻只让一个线程真正算数

GIL 全称全局解释器锁,你把它想成一间厨房里只有一把炒勺:哪怕雇了 8 个厨师(8 个线程),勺只有一把,同一时刻只能一个人拿着炒,其余 7 个在旁边干等接力。它是 Python 解释器天生的设计,规定任一时刻只准一个线程执行字节码。这就是为什么本节里多线程跑纯计算回测耗时 10.24 秒、比单进程 2.49 秒还慢——大家抢同一把勺,谁也快不了,还多了来回切换的开销。普通人改不了这把锁,但只要懂了它,就知道什么时候该绕开它。

► 02 CPU 密集 vs IO 密集:在算数,还是在干等

判断该用哪种并发,先看任务是哪一类。CPU 密集是程序一直埋头算数,比如参数扫描回测、给几千只票算指标,机器一刻不停;IO 密集是程序大部分时间在干等,比如等网络回数据、等磁盘读文件,这期间机器其实闲着。本节用一道一亿次累加代表 CPU 密集、用等 0.2 秒代表 IO 密集。这两类活对付 GIL 的招数正好相反:同样是多线程,在 CPU 密集上慢到 10.24 秒,在 IO 密集上却把 2.0 秒压到 0.2 秒、快了近 10 倍。分错这一刀,后面全白搭。

► 03 多线程适合等待,不适合算数

多线程是在一个进程里同时开好几条执行线、共享同一份内存。关键在于:一条线在干等(比如等网络)的时候,它会主动让出那把炒勺,别的线趁机拿去用。所以 IO 密集特别香——一条线在等,另几条同时在等各自的网络,大家1 块儿等,整体就快了,本节模拟等网络从 2.0 秒压到 0.2 秒就是这个道理。可碰上纯算数,大家都不肯松勺、抢着算,多线程反而添乱,本节多线程跑回测 10.24 秒比单进程还慢,正是老温栽的那个坑。

► 04 多进程绕开 GIL:真正用满多核

多进程是干脆开好几个完全独立的 Python 程序,每个有自己的一把炒勺、自己的一份内存,互不干扰,于是好几个核能真正同时炒菜——这才是用满多核、专治 CPU 密集的正解。本节多进程把回测从 2.49 秒压到 0.30 秒、快了 8.34 倍,而结果和单进程一字不差(True)。代价是开新进程有启动开销、进程间不共享内存数据得打包来回传,所以它只适合单块就要算挺久的大活;碎活几毫秒一个,传数据的功夫比省下的还多,反而亏。

► 05 进程数越多越快,但收益递减

进程不是开得越多越好。本节把进程数从 1 加到 2、4、8,耗时从 2.41 秒一路降到 1.20、0.69、0.46 秒,加速约 1、2.01、3.51、5.23 倍,但你看后半段——从 4 加到 8 提升明显变缓,这就是收益递减。原因有二:一是物理核数有限,进程开得比核还多只能排队;二是每开一个进程都要启动、都要打包传数据,本身就耗时。所以 max_workers 设在电脑物理核数附近最划算,无脑开几百个还会把内存吃光、把电脑卡死,纯属浪费。

05 · REAL MARKETS · 真实市场案例

A 股 · 港股 · 美股 · 加密 全覆盖

// 每个案例都有具体标的和数字,方便你立刻验证

批量回测	sh.600030	给一篮子 A 股(中信证券领衔)各做一遍参数扫描回测,这是典型 CPU 密集任务,用 ProcessPoolExecutor 多进程才能成倍提速
数据拉取		同时拉几十只票的行情数据,大部分时间在等网络,是 IO 密集任务,用 ThreadPoolExecutor 多线程或异步就能并行等待、大幅缩短总耗时
全市场扫描		给沪深几千只票同时算因子打分,纯计算,单核排队要很久,用多进程切成几份分给多核同时算,才能在收盘后短时间内出结果
进程数选择		max_workers 设成你电脑的物理核数附近最划算,设太多进程会因启动和数据传输开销反而变慢,要实测找到最优值

这些坑你不主动避 · 迟早会主动找上你

// 每个坑都附了避坑动作,而不是只描述现象

△ 01

CPU 密集任务开多线程,白忙一场

这是最常见的误区,老温就栽在这。纯计算的活,比如批量回测、给几千只票算指标,开再多线程也只是排队轮流用那一个核,因为 GIL 同一时刻只放一个线程算数,耗时几乎不变。正解是用 ProcessPoolExecutor 多进程,开好几个独立的 python 各占一个核同时算。

△ 02

小任务用多进程,启动开销反而拖慢

多进程不是万能药。开一个新进程本身有启动开销,进程之间还要把数据打包来回传。如果每个小任务才几毫秒,这些开销加起来比省下的计算时间还多,结果比单进程还慢。多进程只适合那种单块就要算挺久的大活,小碎活别用。

△ 03

worker 函数写成闭包或 lambda,没法 pickle

多进程要把你的函数打包(pickle)发给子进程去跑,所以那个被并行调用的函数必须是模块顶层的普通函数,不能是写在别的函数里面的闭包,也不能是 lambda 匿名函数,否则会报 pickle 错误跑不起来。本节的 backtest_scan 就是特意放在模块顶层的。

△ 04

多进程之间想共享一个变量,改了对方看不见

多线程共享同一份内存,而多进程各有各的独立内存。新手常以为在主进程里改个全局变量,子进程就能看到,其实完全看不见,数据只能靠返回值或专门的共享机制传递。想在进程间传结果,就老老实实用 map 收集返回值,别指望共享普通变量。

△ 05

无脑开几百个进程,把内存撑爆

进程不是开得越多越好。每个进程都要复制一份数据、占1 块内存,开几百个进程会瞬间把内存吃光导致电脑卡死甚至崩溃,而且超过物理核数后再加也不会更快。

max_workers 设在电脑物理核数附近最合理,本节的进程数实验就专门演示了收益递减这件事。

并发提速的几条规矩

// 按这套规则走,你的执行力会立刻拉到中等机构水平

- 01 先判断任务类型:一直在算数是 CPU 密集,大部分时间在干等是 IO 密集
- 02 口诀:算数开多进程(ProcessPoolExecutor),等待开多线程(ThreadPoolExecutor)
- 03 CPU 密集千万别指望多线程提速,GIL 会让它排队轮流用一个核
- 04 并行的 worker 函数必须是模块顶层普通函数,不能是闭包或 lambda(要能 pickle)
- 05 进程数设在物理核数附近,别无脑开几百个把内存撑爆,实测找最优

► **纪律 > 灵感:** 量化做久的人都明白,赚钱的不是某一次绝妙判断,而是把规则坚持执行 1000 次。打印这一页,贴在你电脑边。

把这节课带走的几件事

// 打印或截图这一页, 3 天后再看一遍效果最好

- 01 GIL 是 Python 的一把锁:同一时刻只准一个线程真正占用 CPU 算数。
- 02 任务分两类:一直在算数叫 CPU 密集,大部分时间在干等叫 IO 密集。
- 03 CPU 密集开多线程几乎不提速(卡在 GIL),必须用多进程绕开它、用满多核。
- 04 IO 密集开多线程很管用,一个线程在等时让出锁,别的线程趁机干活。
- 05 concurrent.futures 给你 ThreadPoolExecutor 和 ProcessPoolExecutor,一行 map 调度,改一个词就切换。
- 06 口诀:算数开多进程,等待开多线程;worker 要放模块顶层,进程数别开太多。

08 · SELF-CHECK · 自测题

答不上来的回看视频对应段落

// 这几道题都没标准答案,目的是逼你自己组织语言讲清楚

Q01 用『一间厨房只有一把炒勺』打比方,解释什么是 GIL,以及它为什么让多线程算数没用。

Q02 什么是 CPU 密集任务、什么是 IO 密集任务?各举一个量化里的例子,并说该用多线程还是多进程。

Q03 老温把回测从多线程改成多进程只换了一个词就快了好几倍,这背后是什么道理?

Q04 为什么并行用的 worker 函数不能写成闭包或 lambda?进程数是不是开得越多越好?

09 · NEXT STEP · 下一步

继续往前走

// 看完这节,下面是延伸方向 + 明天预告

推荐阅读 · FURTHER READING

// 听完今天的内容还想往深里挖的话

- ▶ Beazley & Jones 《Python Cookbook》(2013/O'Reilly)– 第 12 章并发编程,线程进程协程讲得最实用的工具书。
- ▶ Ramalho 《Fluent Python》(2022/O'Reilly)– 并发章把 GIL、concurrent.futures、多进程多线程的取舍讲得透彻又深入。
- ▶ Gorelick & Ozsvald 《High Performance Python》(2020/O'Reilly)– 专讲 Python 提速,多核并行与绕开 GIL 的实战手册。
- ▶ Python 官方文档 concurrent.futures(docs.python.org)– ThreadPoolExecutor / ProcessPoolExecutor / map 的权威函数手册。
- ▶ David Beazley 《Understanding the Python GIL》(2010/PyCon 演讲)– 把 GIL 内部机制讲透的经典分享,理解为什么多线程算数没用。

NEXT LECTURE · 下一节预告

DAY 054 Numba JIT 加速

这一节我们靠多进程把多核都用上了,但每个核里那段双层循环回测本身还是慢。下一节请出加速神器 Numba,只在函数上加一行 JIT 装饰器,就能把这种逐日循环的回测代码提速几十倍,让纯 Python 跑出接近 C 语言的速度。